

Informe de pruebas de caja negra

GraphRAG Auditor — API y flujo del documento

Suite: tests/blackbox — 33 pruebas

Fecha de ejecución: 12 de julio de 2026

1. Alcance de la prueba

Una prueba de caja negra ejercita el sistema por su interfaz pública, sin conocer ni tocar su implementación: se le entrega una petición y se observa la respuesta, y el criterio de acierto procede de la especificación, no del código. Frente a las pruebas unitarias, que verifican que cada pieza calcula bien, estas verifican que el sistema hace lo que promete a quien lo consume, y son las únicas capaces de detectar que dos piezas correctas están mal conectadas.

La interfaz pública del sistema es su API HTTP. La suite entra siempre por ella —por la URL— y comprueba únicamente el código de estado y el cuerpo de la respuesta. El alcance cubre dos niveles:

Nivel	Qué verifica	Técnicas de diseño de casos	Pruebas
Contrato de la API	Disponibilidad del servicio, validación de la carga de documentos y respuesta ante recursos inexistentes	Partición de equivalencia, valores límite y adivinación de errores	18
Flujo y máquina de estados	Ciclo de vida del documento (procesando → revisión pendiente → listo para extracción) y transiciones inválidas	Prueba de transición de estados	15

Queda fuera del alcance la ejecución de la auditoría semántica completa, que invoca al modelo de lenguaje una vez por cita. De ella se comprueban sus guardas —qué ocurre cuando se pide auditar un documento sin citas, o consultar veredictos que aún no existen—, que es lo que corresponde a este nivel de prueba. La medición de la eficacia del sistema (Kappa de Cohen y F1 frente a un experto) es una evaluación distinta, con criterios de umbral, y no forma parte de este informe.

Cada prueba se documenta con su descripción, su código, la petición enviada, la respuesta recibida y su resultado. Peticiones y respuestas son las de la ejecución real: el cliente de pruebas las registró durante la corrida.

2. Documento de prueba

El sujeto de las pruebas de flujo es el mismo documento real empleado en las pruebas unitarias: la tesis del autor del proyecto, en PDF. La suite lo carga por la API y recorre con él el ciclo de vida completo, de modo que los datos que se comprueban son los que el sistema extrajo de un documento académico auténtico.

Atributo	Valor observado en la ejecución
----------	---------------------------------

Archivo	Informe de tesis_AlcaldLavadoMatias (1).pdf
Páginas	67
Identificador asignado por el sistema	7f134334-8713-4f8a-84ff-cc45a55ed143
Citas extraídas	44
Referencias extraídas	40
Citas vinculadas a su referencia	42 de 44
Nodos del grafo de conocimiento	264 (180 autores, 40 referencias, 44 citas)
Relaciones del grafo	306
Estado alcanzado	listo_extraccion (revisión confirmada)

Las pruebas de contrato emplean además archivos contruidos para cada clase de entrada: un PDF legible de exactamente el tamaño máximo admitido, otro de un byte por encima del límite, un archivo vacío, un archivo con extensión no admitida y un PDF con cabecera válida pero cuerpo ilegible.

3. Dependencias y aislamiento

A diferencia de las pruebas unitarias, aquí no se sustituye ni se parchea nada. La suite se ejecuta contra los servicios reales configurados en el entorno del sistema, porque el objeto de la prueba es precisamente el conjunto en funcionamiento: si el grafo, el almacén vectorial o el modelo de lenguaje no responden como el sistema espera, la prueba debe verlo.

Dependencia	Tratamiento	Uso durante la ejecución
Neo4j (grafo de conocimiento)	Real	Escritura y lectura de las citas, referencias y autores del documento cargado
Supabase Postgres y pgvector	Real	Persistencia del progreso del pipeline y almacén de fragmentos
Supabase Storage	Real	Subida y descarga del PDF original
Modelo de lenguaje (Groq)	Real	Extracción de citas y referencias durante el pipeline de carga
CrossRef y Unpaywall	No invocados	La verificación externa no se ejecuta: solo se comprueban sus guardas
Interfaz web (React + Vite)	Real	Conducida por un navegador automatizado para las capturas de la sección 9

La consecuencia de ejecutar sin aislamiento es que la suite deja huella: cada corrida carga un documento y escribe sus nodos en el grafo. El documento de esta ejecución es el identificado en la sección 2, y puede eliminarse cuando ya no se necesite como evidencia.

4. Entorno de ejecución

Elemento	Valor
Sistema operativo	Linux 6.6.87.2-microsoft-standard-WSL2 (WSL2 sobre Windows)
Intérprete	Python 3.11.15
Framework de la API	FastAPI 0.136.1 sobre Starlette
Framework de pruebas	pytest 8.3.4

Cliente HTTP	TestClient de FastAPI, sobre la aplicación real
Interfaz web	Vite en http://localhost:5173, contra la API en http://127.0.0.1:8000
Navegador de las capturas	Chromium 149 (Playwright), 1440 × 900, sin interfaz gráfica
Comando ejecutado	python -m pytest tests/blackbox -v

El cliente de pruebas se configura para no propagar las excepciones del servidor: un fallo no controlado debe observarse como lo observaría un cliente real, es decir, como una respuesta 500, y no como una excepción dentro de la prueba. Esta decisión es la que permite documentar el defecto de la sección 8 con su respuesta real.

5. Resumen de resultados

Métrica	Valor
Pruebas ejecutadas	33
Pruebas superadas (PASSED)	32
Pruebas fallidas (FAILED)	1
Tasa de éxito	97 %
Tiempo total de ejecución	3 min 59 s (238,96 s)
Defectos del sistema detectados	1 (sección 8)

El desglose por nivel y clase de prueba es el siguiente:

Nivel	Clase de prueba	Pruebas	Superadas	Fallidas
Contrato de la API	TestDisponibilidadDelServicio	3	3	0
Contrato de la API	TestCargaDeDocumento	7	6	1
Contrato de la API	TestDocumentoInexistente	8	8	0
Flujo del documento	TestCicloDeVidaDelDocumento	8	8	0
Flujo del documento	TestTransicionesInvalidas	7	7	0
Total		33	32	1

La salida final del ejecutor fue:

```
===== 1 failed, 32 passed, 8 warnings in 238.96s (0:03:58) =====
```

La única prueba fallida no revela un error en la prueba, sino un defecto del sistema: se describe en la sección 8. Las 32 restantes confirman que la API cumple su contrato y que la máquina de estados del documento impide las transiciones que debe impedir.

6. Fixtures compartidas

Las fixtures se definen en tests/blackbox/confest.py. La carga del documento es costosa — dispara el pipeline completo de extracción—, de modo que se realiza una sola vez por sesión y su resultado se comparte entre todas las pruebas de flujo.

Fixture	Ámbito	Descripción
api	session	Cliente HTTP sobre la aplicación real. Registra cada petición y

		cada respuesta, y observa los fallos del servidor como respuestas 500 en lugar de propagarlos como excepciones.
pdf_tesis	session	Bytes del PDF real de la tesis, empleado como documento de prueba.
pdf_en_el_limite	session	PDF legible con capa de texto, construido para pesar exactamente el máximo admitido. Es la entrada del caso de valor límite.
documento_inexistente	session	Identificador con formato válido que no corresponde a ningún documento. Es la entrada de los casos de recurso no encontrado.
documento	session	Carga el PDF por la API y espera, consumiendo el stream de progreso, a que el pipeline llegue al estado de revisión pendiente. Es el sujeto de las pruebas de flujo.

El código de las fixtures compartidas es el siguiente:

```

"""
Pruebas de caja negra: el sistema se ejercita por su interfaz pública (la API
HTTP), sin conocer ni tocar sus interioridades. No se parchea nada: los
endpoints trabajan contra los servicios reales configurados en backend/.env
(Neo4j, Supabase, Storage y el LLM).

El cliente de pruebas registra cada petición y cada respuesta, de modo que la
evidencia del informe es la de la ejecución real.
"""

import json
import os
import time
import uuid
from pathlib import Path

import pytest
from fastapi.testclient import TestClient

from app.main import app

PDF_TESIS = Path(__file__).parent.parent / "unit" / "Informe de
tesis_AlcaldeLavadoMatias (1).pdf"
EVIDENCIA = Path(os.environ.get("EVIDENCIA_JSON", Path(__file__).parent /
"evidencia.json"))
LIMITE_MB = 10
ESPERA_PIPELINE_S = 900

_evidencia: dict = {}
_actual = {"id": None}

def _corto(texto: str, limite: int = 900) -> str:
    if len(texto) > limite:
        return texto[:limite] + f" ... [{len(texto)} caracteres en total]"
    return texto

class ClienteRegistrado:
    """Cliente HTTP que anota lo que envía y lo que recibe."""

    def __init__(self, cliente: TestClient):
        self._cliente = cliente

    def request(self, metodo: str, url: str, *, descripcion_entrada: str = "",

```

```

**kwargs):
    respuesta = self._cliente.request(metodo, url, **kwargs)
    try:
        cuerpo = json.dumps(respuesta.json(), ensure_ascii=False, indent=1)
    except Exception:
        cuerpo = f"<{respuesta.headers.get('content-type', 'desconocido')}>,"
        cuerpo += f"{len(respuesta.content)} bytes>"

    entrada = descripcion_entrada
    if not entrada and "json" in kwargs:
        entrada = json.dumps(kwargs["json"], ensure_ascii=False)

    if _actual["id"]:
        _evidencia.setdefault(_actual["id"], {"intercambios":
[[]]}["intercambios"].append(
            {
                "peticion": f"{metodo.upper()} {url}",
                "cuerpo_enviado": entrada,
                "estado": f"{respuesta.status_code} {respuesta.reason_phrase}",
                "cuerpo_recibido": _corto(cuerpo),
            }
        )
    return respuesta

def get(self, url, **kw):
    return self.request("GET", url, **kw)

def post(self, url, **kw):
    return self.request("POST", url, **kw)

def stream(self, metodo, url, **kw):
    return self._cliente.stream(metodo, url, **kw)

def anotar(self, peticion: str, entrada: str, estado: str, salida: str):
    if _actual["id"]:
        _evidencia.setdefault(_actual["id"], {"intercambios":
[[]]}["intercambios"].append(
            {
                "peticion": peticion,
                "cuerpo_enviado": entrada,
                "estado": estado,
                "cuerpo_recibido": _corto(salida),
            }
        )

@pytest.fixture(scope="session")
def api():
    # raise_server_exceptions=False: un fallo no controlado del servidor debe
    # observarse como lo observaría un cliente real –una respuesta 500– y no
    # como una excepción dentro de la prueba.
    with TestClient(app, raise_server_exceptions=False) as cliente:
        yield ClienteRegistrado(cliente)

@pytest.fixture(scope="session")
def pdf_tesis() -> bytes:
    if not PDF_TESIS.exists():
        pytest.skip("No se encontró el PDF de prueba.")
    return PDF_TESIS.read_bytes()

```

```

@pytest.fixture(scope="session")
def pdf_en_el_limite() -> bytes:
    """PDF legible de exactamente el tamaño máximo admitido, para el valor
    límite."""
    import fitz

    documento = fitz.open()
    pagina = documento.new_page()
    pagina.insert_text((72, 72), "Documento de prueba para el valor limite de
    tamaño.")
    pagina.insert_text((72, 96), "Contiene capa de texto, como exige el sistema.")
    minimo = documento.tobytes()
    documento.close()

    objetivo = LIMITE_MB * 1024 * 1024
    relleno = b"\n%" + b" " * (objetivo - len(minimo) - 2) # comentario PDF: no
    altera el contenido
    return minimo + relleno

@pytest.fixture(scope="session")
def documento_inexistente() -> str:
    """Identificador con formato válido que no corresponde a ningún documento."""
    return str(uuid.uuid4())

@pytest.fixture(scope="session")
def documento(api, pdf_tesis) -> dict:
    """
    Carga el documento de prueba y espera a que el pipeline termine la extracción.
    Es el sujeto de las pruebas de flujo: una sola carga para toda la sesión.
    """
    respuesta = api._cliente.post(
        "/api/v1/ingesta/cargar",
        files={"archivo": (pdf_tesis, "application/pdf")},
    )
    assert respuesta.status_code == 200, respuesta.text
    datos = respuesta.json()
    datos["_carga"] = {
        "peticion": "POST /api/v1/ingesta/cargar",
        "cuerpo_enviado": f"multipart/form-data; archivo=tesis.pdf
        (<{len(pdf_tesis)} bytes>)",
        "estado": f"{respuesta.status_code} {respuesta.reason_phrase}",
        "cuerpo_recibido": json.dumps(datos, ensure_ascii=False, indent=1),
    }

    documento_id = datos["documento_id"]
    eventos = []
    inicio = time.time()
    with api.stream("GET", f"/api/v1/ingesta/{documento_id}/progreso") as flujo:
        for linea in flujo.iter_lines():
            if linea.startswith("data: "):
                eventos.append(json.loads(linea[6:]))
                if eventos[-1]["estado"] in ("revision_pendiente", "completado",
                "error"):
                    break
            if time.time() - inicio > ESPERA_PIPELINE_S:
                pytest.fail("El pipeline no terminó dentro del tiempo previsto.")

    datos["_eventos"] = eventos

```

```

    datos["_segundos"] = round(time.time() - inicio, 1)
    assert eventos and eventos[-1]["estado"] == "revisión_pendiente", eventos[-1] if
eventos else "sin eventos"
    return datos

def pytest_runtest_call(item):
    _actual["id"] = item.nodeid

def pytest_runtest_logreport(report):
    if report.when == "call":
        _evidencia.setdefault(report.nodeid, {"intercambios": []})["resultado"] =
report.outcome.upper()
        _actual["id"] = None

def pytest_sessionfinish(session, exitstatus):
    EVIDENCIA.write_text(json.dumps(_evidencia, indent=1, ensure_ascii=False),
encoding="utf-8")
    print(f"\n[evidencia] {len(_evidencia)} pruebas registradas en {EVIDENCIA}")

```

7. Detalle de las pruebas

Se documenta cada prueba con su descripción, su código, la petición enviada al sistema, la respuesta que este devolvió y el resultado de la comprobación. Cuando una prueba realiza varias peticiones, se numeran en orden. Los cuerpos muy extensos se abrevian indicando su tamaño real.

7.1. Contrato de la API

Archivo de pruebas	Pruebas
tests/blackbox/test_contrato_api.py	13

Contrato de la API: lo que el sistema promete a quien lo consume.

Los casos de la carga de documentos se derivan por partición de equivalencia (una clase de entrada, un caso) y por valores límite sobre el tamaño máximo admitido. Los del documento inexistente comprueban que el sistema responde con un error explícito y nunca con un fallo interno.

7.1.1. TestDisponibilidadDelServicio

test_el_servicio_responde_que_esta_sano

Descripción. El endpoint de salud responde 200 y declara el servicio operativo. Es la comprobación previa a cualquier otra: si esto falla, el resto de resultados no significan nada.

Código.

```

def test_el_servicio_responde_que_esta_sano(self, api):
    r = api.get("/health")
    assert r.status_code == 200
    assert r.json()["status"] == "ok"

```

Petición.

```
GET /health
```

Respuesta.

```
200 OK
{
  "status": "ok"
}
```

Resultado. PASSED**test_la_raiz_identifica_el_servicio_y_su_version**

Descripción. La raíz de la API identifica el servicio por su nombre y declara su estado operativo.

Código.

```
def test_la_raiz_identifica_el_servicio_y_su_version(self, api):
    r = api.get("/")
    assert r.status_code == 200
    assert r.json()["nombre"] == "GraphRAG Auditor API"
    assert r.json()["estado"] == "operativo"
```

Petición.

```
GET /
```

Respuesta.

```
200 OK
{
  "nombre": "GraphRAG Auditor API",
  "version": "0.1.0",
  "estado": "operativo",
  "docs": "/docs"
}
```

Resultado. PASSED**test_el_contrato_openapi_esta_publicado**

Descripción. El contrato OpenAPI está publicado y declara las rutas de carga y de auditoría: el consumidor puede descubrir la API sin leer el código.

Código.

```
def test_el_contrato_openapi_esta_publicado(self, api):
    r = api.get("/openapi.json")
    assert r.status_code == 200
    rutas = r.json()["paths"]
    assert "/api/v1/ingesta/cargar" in rutas
    assert "/api/v1/auditoria/{documento_id}/auditar" in rutas
```

Petición.

```
GET /openapi.json
```

Respuesta.

```
200 OK
{
  "openapi": "3.1.0",
  "info": {
    "title": "GraphRAG Auditor API",
```

```

    "description": "Sistema de auditoría semántica de citas y referencias bibliográficas
    APA 7ma
    edición.",
    "version": "0.1.0"
  },
  "paths": {
    "/api/v1/ingesta/cargar": {
      "post": {
        "tags": [
          "Ingesta"
        ],
        "summary": "HU-001: Cargar documento PDF",
        "description": "Recibe un PDF, lo valida, guarda y arranca la fase 1 del pipeline
        automáticamente.",
        "operationId": "cargar_pdf_api_v1_ingesta_cargar_post",
        "requestBody": {
          "content": {
            "multipart/form-data": {
              "schema": {
                "$ref": "#/components/schemas/Body_cargar_pdf_api_v1_ingesta_cargar_post"
              }
            }
          }
        },
        "required": true
      },
      "responses": {
        "200": {
          "description": "Successful Response",
          "content": {
            "application/json": {
              "schema": {
                "$ref": "#/compone ... [83418 caracteres en total]"
              }
            }
          }
        }
      }
    }
  }
}

```

Resultado. PASSED

7.1.2. TestCargaDeDocumento

Partición de equivalencia y valores límite sobre POST /ingesta/cargar.

test_una_peticion_sin_archivo_es_rechazada

Descripción. Una petición de carga sin archivo adjunto se rechaza con 422 por validación de la propia API, sin llegar al servicio.

Código.

```

def test_una_peticion_sin_archivo_es_rechazada(self, api):
    r = api.request("POST", "/api/v1/ingesta/cargar", descripcion_entrada="(sin
    archivo)")
    assert r.status_code == 422

```

Petición.

```

POST /api/v1/ingesta/cargar
cuerpo: (sin archivo)

```

Respuesta.

```

422 Unprocessable Entity
{
  "detail": [
    {
      "type": "missing",
      "loc": [

```

```

        "body",
        "archivo"
    ],
    "msg": "Field required",
    "input": null
}
]
}

```

Resultado. PASSED

test_un_archivo_que_no_es_pdf_es_rechazado

Descripción. Clase de equivalencia «archivo con extensión no admitida»: se rechaza con 422, código FORMATO_INVALIDO, y la respuesta incluye una acción sugerida para el usuario.

Código.

```

def test_un_archivo_que_no_es_pdf_es_rechazado(self, api, pdf_tesis):
    r = api.post(
        "/api/v1/ingesta/cargar",
        files={"archivo": ("tesis.docx", pdf_tesis, "application/msword")},
        descripcion_entrada=f"archivo=tesis.docx (<{len(pdf_tesis)} bytes>)",
    )
    assert r.status_code == 422
    assert r.json()["detail"]["codigo"] == "FORMATO_INVALIDO"
    assert r.json()["detail"]["accion_sugerida"]

```

Petición.

```

POST /api/v1/ingesta/cargar
cuerpo: archivo=tesis.docx (<730941 bytes>)

```

Respuesta.

```

422 Unprocessable Entity
{
  "detail": {
    "codigo": "FORMATO_INVALIDO",
    "mensaje": "El archivo no es un PDF válido.",
    "accion_sugerida": "Sube un archivo con extensión .pdf"
  }
}

```

Resultado. PASSED

test_un_pdf_con_contenido_corrupto_es_rechazado

Descripción. Clase de equivalencia «contenido que no es un PDF»: un archivo con nombre .pdf pero sin estructura PDF se rechaza con 422 y código PDF_CORRUPTO.

Código.

```

def test_un_pdf_con_contenido_corrupto_es_rechazado(self, api):
    contenido = b"esto no es un PDF"
    r = api.post(
        "/api/v1/ingesta/cargar",
        files={"archivo": ("tesis.pdf", contenido, "application/pdf")},
        descripcion_entrada=f"archivo=tesis.pdf ({contenido!r})",
    )
    assert r.status_code == 422
    assert r.json()["detail"]["codigo"] == "PDF_CORRUPTO"

```

Petición.

```
POST /api/v1/ingesta/cargar
cuerpo: archivo=tesis.pdf (b'esto no es un PDF')
```

Respuesta.

```
422 Unprocessable Entity
{
  "detail": {
    "codigo": "PDF_CORRUPTO",
    "mensaje": "El archivo no tiene una estructura PDF válida.",
    "accion_sugerida": "Verifica que el archivo no esté dañado e intenta exportarlo nuevamente."
  }
}
```

Resultado. PASSED

test_un_archivo_vacio_es_rechazado

Descripción. Clase de equivalencia «archivo vacío»: cero bytes se rechazan con 422 en lugar de admitirse y fallar más adelante.

Código.

```
def test_un_archivo_vacio_es_rechazado(self, api):
    r = api.post(
        "/api/v1/ingesta/cargar",
        files={"archivo": ("tesis.pdf", b"", "application/pdf")},
        descripcion_entrada="archivo=tesis.pdf (<0 bytes>)",
    )
    assert r.status_code == 422
    assert r.json()["detail"]["codigo"] in ("PDF_CORRUPTO", "ARCHIVO_VACIO")
```

Petición.

```
POST /api/v1/ingesta/cargar
cuerpo: archivo=tesis.pdf (<0 bytes>)
```

Respuesta.

```
422 Unprocessable Entity
{
  "detail": {
    "codigo": "PDF_CORRUPTO",
    "mensaje": "El archivo no tiene una estructura PDF válida.",
    "accion_sugerida": "Verifica que el archivo no esté dañado e intenta exportarlo nuevamente."
  }
}
```

Resultado. PASSED

test_un_archivo_que_supera_el_limite_es_rechazado

Descripción. Valor límite superior: un archivo de exactamente el máximo admitido más un byte se rechaza con 422, código ARCHIVO_DEMASIADO_GRANDE, y el mensaje declara el límite en megabytes.

Código.

```
def test_un_archivo_que_supera_el_limite_es_rechazado(self, api):
    """Valor límite superior: un byte por encima del máximo admitido."""
    contenido = CABECERA_PDF + b"\0" * (LIMITE_MB * 1024 * 1024 -
len(CABECERA_PDF) + 1)
    r = api.post(
        "/api/v1/ingesta/cargar",
        files={"archivo": ("tesis.pdf", contenido, "application/pdf")},
        descripcion_entrada=f"archivo=tesis.pdf (<{len(contenido)} bytes> =
límite + 1)",
    )
    assert r.status_code == 422
    assert r.json()["detail"]["codigo"] == "ARCHIVO_DEMASIADO_GRANDE"
    assert str(LIMITE_MB) in r.json()["detail"]["mensaje"]
```

Petición.

```
POST /api/v1/ingesta/cargar
cuerpo: archivo=tesis.pdf (<10485761 bytes> = límite + 1)
```

Respuesta.

```
422 Unprocessable Entity
{
  "detail": {
    "codigo": "ARCHIVO_DEMASIADO_GRANDE",
    "mensaje": "El archivo pesa 10.0 MB y supera el límite de 10 MB.",
    "accion_sugerida": "Comprime el PDF o divídelo en partes más pequeñas."
  }
}
```

Resultado. PASSED

test_en_el_limite_exacto_el_archivo_se_acepta

Descripción. Valor límite: un PDF legible de exactamente el tamaño máximo admitido se acepta con 200. Demuestra que el límite es inclusivo y que la comprobación no rechaza por exceso en la frontera.

Código.

```
def test_en_el_limite_exacto_el_archivo_se_acepta(self, api, pdf_en_el_limite):
    """Valor límite: exactamente el máximo admitido. El límite es inclusivo."""
    r = api.post(
        "/api/v1/ingesta/cargar",
        files={"archivo": ("limite.pdf", pdf_en_el_limite, "application/pdf")},
        descripcion_entrada=f"archivo=limite.pdf (<{len(pdf_en_el_limite)}
bytes> = límite exacto)",
    )
    assert r.status_code == 200
    assert r.json()["tamano_bytes"] == LIMITE_MB * 1024 * 1024
```

Petición.

```
POST /api/v1/ingesta/cargar
cuerpo: archivo=limite.pdf (<10485760 bytes> = límite exacto)
```

Respuesta.

```
200 OK
{
  "documento_id": "48a7a664-39d4-431d-9612-da7d98ffe416",
  "nombre_archivo": "limite.pdf",
  "tamano_bytes": 10485760,
```

```

    "paginas": 1,
    "estado": "procesando",
    "mensaje": "Documento recibido. Extracción iniciada."
}

```

Resultado. PASSED

test_un_pdf_con_cabecera_valida_pero_cuerpo_danado_es_rechazado

Descripción. Un PDF con cabecera %PDF correcta pero cuerpo ilegible —una descarga interrumpida, una exportación a medias— debe rechazarse con 422 y código PDF_CORRUPTO, como cualquier otro archivo no procesable. Es el caso que revela el defecto descrito en la sección 8.

Código.

```

def test_un_pdf_con_cabecera_valida_pero_cuerpo_danado_es_rechazado(self, api):
    """
    Un PDF truncado o dañado (cabecera correcta, contenido ilegible) es el
    caso más frecuente en la práctica: una descarga interrumpida, una
    exportación a medias. Debe rechazarse con 422 y código PDF_CORRUPTO,
    igual que cualquier otro archivo no procesable.
    """
    contenido = CABECERA_PDF + b"\0" * 4096
    r = api.post(
        "/api/v1/ingesta/cargar",
        files={"archivo": ("danado.pdf", contenido, "application/pdf")},
        descripcion_entrada=f"archivo=danado.pdf (<{len(contenido)} bytes>,
        cabecera %PDF y cuerpo ilegible)",
    )
    assert r.status_code == 422, f"Se obtuvo {r.status_code} en lugar de 422."
    assert r.json()["detail"]["codigo"] == "PDF_CORRUPTO"

```

Petición.

```

POST /api/v1/ingesta/cargar
cuerpo: archivo=danado.pdf (<4105 bytes>, cabecera %PDF y cuerpo ilegible)

```

Respuesta.

```

500 Internal Server Error
<text/plain; charset=utf-8, 21 bytes>

```

Resultado. FAILED

7.1.3. TestDocumentoInexistente

Todo recurso pedido sobre un documento que no existe da un error explícito.

test_devuelve_404_con_codigo_de_error_propio

Descripción. Pedir el recurso sobre un documento que no existe devuelve 404 con un código de error propio del sistema y una acción sugerida, en lugar de un error genérico o un fallo interno.

Código.

```

def test_devuelve_404_con_codigo_de_error_propio(self, api, documento_inexistente,
ruta, codigo):
    r = api.get(ruta.format(d=documento_inexistente))
    assert r.status_code == 404
    assert r.json()["detail"]["codigo"] == codigo
    assert r.json()["detail"]["accion_sugerida"]

```

/api/v1/ingesta/{d}/markdown-MARKDOWN_NO_ENCONTRADO

Petición.

GET /api/v1/ingesta/850887b8-9e65-4a64-ad37-aef1a9895a91/markdown

Respuesta.

```
404 Not Found
{
  "detail": {
    "codigo": "MARKDOWN_NO_ENCONTRADO",
    "mensaje": "No se encontró el texto procesado del documento.",
    "accion_sugerida": "Vuelve a cargar el archivo PDF."
  }
}
```

Resultado. PASSED

/api/v1/ingesta/{d}/pdf-DOCUMENTO_NO_ENCONTRADO

Petición.

GET /api/v1/ingesta/850887b8-9e65-4a64-ad37-aef1a9895a91/pdf

Respuesta.

```
404 Not Found
{
  "detail": {
    "codigo": "DOCUMENTO_NO_ENCONTRADO",
    "mensaje": "No se encontró el PDF del documento.",
    "accion_sugerida": "Vuelve a cargar el archivo PDF."
  }
}
```

Resultado. PASSED

/api/v1/ingesta/{d}/estructura-DOCUMENTO_NO_ENCONTRADO

Petición.

GET /api/v1/ingesta/850887b8-9e65-4a64-ad37-aef1a9895a91/estructura

Respuesta.

```
404 Not Found
{
  "detail": {
    "codigo": "DOCUMENTO_NO_ENCONTRADO",
    "mensaje": "No se encontró el documento.",
    "accion_sugerida": "Vuelve a cargar el archivo PDF."
  }
}
```

Resultado. PASSED

/api/v1/grafos/{d}/referencias-REFERENCIAS_NO_ENCONTRADAS

Petición.

GET /api/v1/grafos/850887b8-9e65-4a64-ad37-aef1a9895a91/referencias

Respuesta.

```
404 Not Found
{
  "detail": {
    "codigo": "REFERENCIAS_NO_ENCONTRADAS",
    "mensaje": "No se encontraron referencias para este documento.",
    "accion_sugerida": "Verifica que la auditoría se haya completado."
  }
}
```

Resultado. PASSED

/api/v1/auditoria/{d}/veredictos-VEREDICTOS_NO_ENCONTRADOS

Petición.

```
GET /api/v1/auditoria/850887b8-9e65-4a64-ad37-aef1a9895a91/veredictos
```

Respuesta.

```
404 Not Found
{
  "detail": {
    "codigo": "VEREDICTOS_NO_ENCONTRADOS",
    "mensaje": "No hay veredictos calculados para este documento.",
    "accion_sugerida": "Ejecuta primero POST /auditoria/{documento_id}/auditar."
  }
}
```

Resultado. PASSED

/api/v1/auditoria/{d}/metricas-METRICAS_NO_ENCONTRADAS

Petición.

```
GET /api/v1/auditoria/850887b8-9e65-4a64-ad37-aef1a9895a91/metricas
```

Respuesta.

```
404 Not Found
{
  "detail": {
    "codigo": "METRICAS_NO_ENCONTRADAS",
    "mensaje": "No hay métricas RAGAS calculadas para este documento.",
    "accion_sugerida": "Ejecuta primero POST /auditoria/{documento_id}/evaluar-ragas."
  }
}
```

Resultado. PASSED

test_las_colecciones_vacias_no_son_un_error

Descripción. Pedir las citas de un documento sin citas devuelve una colección vacía con 200: la ausencia de elementos no es un error, y el consumidor no tiene que distinguir entre «vacío» y «no encontrado».

Código.

```
def test_las_colecciones_vacias_no_son_un_error(self, api, documento_inexistente):
    """Pedir las citas de un documento sin citas devuelve una colección vacía,
    no un 404."""
    r = api.get(f"/api/v1/grafos/{documento_inexistente}/citas")
    assert r.status_code == 200
    assert r.json()["total_citas"] == 0
```

Petición.

```
GET /api/v1/grafos/850887b8-9e65-4a64-ad37-aef1a9895a91/citas
```

Respuesta.

```
200 OK
{
  "documento_id": "850887b8-9e65-4a64-ad37-aef1a9895a91",
  "total_citas": 0,
  "citas_parenteticas": 0,
  "citas_narrativas": 0,
  "citas": [],
  "advertencia": "No se detectaron citas APA en el documento."
}
```

Resultado. PASSED

test_un_identificador_con_inyeccion_no_rompe_el_servicio

Descripción. Un identificador que contiene una consulta Cypher no rompe el servicio ni devuelve datos del grafo: el identificador se trata como dato y no como consulta.

Código.

```
def test_un_identificador_con_inyeccion_no_rompe_el_servicio(self, api):
    """El identificador se trata como dato, no como consulta: ni 500 ni fuga de
    datos."""
    malicioso = "x' OR 1=1 RETURN n //"
    r = api.get(f"/api/v1/grafos/{malicioso}/citas")
    assert r.status_code < 500
    assert "cita" not in r.text.lower()
```

Petición.

```
GET /api/v1/grafos/x' OR 1=1 RETURN n ///citas
```

Respuesta.

```
404 Not Found
{
  "detail": "Not Found"
}
```

Resultado. PASSED

7.2. Flujo del documento y máquina de estados

Archivo de pruebas	Pruebas
tests/blackbox/test_flujo_documento.py	15

Flujo del documento y máquina de estados.

Se carga el documento de prueba real y se recorre el ciclo de vida por la API: PROCESANDO → REVISION_PENDIENTE → LISTO_EXTRACCION. Además de las transiciones válidas, se comprueban las inválidas, que son las que un usuario provoca sin querer y las que no deben producir nunca un fallo interno.

La auditoría completa no se ejecuta aquí: se comprueban sus guardas, que es lo que corresponde a este nivel de prueba.

7.2.1. TestCicloDeVidaDelDocumento

test_la_carga_devuelve_identificador_paginas_y_estado_inicial

Descripción. La carga del documento de prueba devuelve su identificador, el número real de páginas (67) y el estado inicial del pipeline.

Código.

```
def test_la_carga_devuelve_identificador_paginas_y_estado_inicial(self, api,
    documento):
    c = documento["_carga"]
    api.anotar(c["peticion"], c["cuerpo_enviado"], c["estado"],
    c["cuerpo_recibido"])
    assert documento["paginas"] == 67
    assert documento["estado"] == "procesando"
    assert documento["documento_id"]
```

Petición.

```
POST /api/v1/ingesta/cargar
cuerpo: multipart/form-data; archivo=tesis.pdf (<730941 bytes>)
```

Respuesta.

```
200 OK
{
  "documento_id": "7f134334-8713-4f8a-84ff-cc45a55ed143",
  "nombre_archivo": "tesis.pdf",
  "tamano_bytes": 730941,
  "paginas": 67,
  "estado": "procesando",
  "mensaje": "Documento recibido. Extracción iniciada."
}
```

Resultado. PASSED

test_el_stream_de_progreso_recorre_el_pipeline_hasta_la_revision

Descripción. El stream de progreso emite eventos con porcentaje creciente y termina en el estado revision_pendiente, que es donde el pipeline cede el control al usuario.

Código.

```
def test_el_stream_de_progreso_recorre_el_pipeline_hasta_la_revision(self, api,
    documento):
    eventos = documento["_eventos"]
    api.anotar(
        f"GET /api/v1/ingesta/{documento['documento_id']}/progreso (stream
    SSE)",
        "(sin cuerpo)",
        "200 OK text/event-stream",
        f"{len(eventos)} eventos en {documento['_segundos']} s. Último:\n"
        + json.dumps(eventos[-1], ensure_ascii=False, indent=1),
    )
    assert [e["estado"] for e in eventos][-1] == "revision_pendiente"
    assert [e["porcentaje"] for e in eventos] == sorted(e["porcentaje"] for e in
    eventos)
```

Petición.

```
GET /api/v1/ingesta/7f134334-8713-4f8a-84ff-cc45a55ed143/progreso (stream SSE)
cuerpo: (sin cuerpo)
```

Respuesta.

```
200 OK text/event-stream
8 eventos en 159.1 s. Último:
{
  "documento_id": "7f134334-8713-4f8a-84ff-cc45a55ed143",
  "estado": "revision_pendiente",
  "porcentaje": 100,
  "mensaje_progreso": "Extracción completada. Revisa y confirma las citas y referencias
antes de
continuar.",
  "citas_encontradas": 44,
  "error": null
}
```

Resultado. PASSED

test_la_estructura_detectada_incluye_la_seccion_de_referencias

Descripción. La estructura detectada del documento declara que tiene sección de referencias bibliográficas.

Código.

```
def test_la_estructura_detectada_incluye_la_seccion_de_referencias(self, api,
documento):
    r = api.get(f"/api/v1/ingesta/{documento['documento_id']}/estructura")
    assert r.status_code == 200
    assert r.json()["tiene_seccion_referencias"] is True
```

Petición.

```
GET /api/v1/ingesta/7f134334-8713-4f8a-84ff-cc45a55ed143/estructura
```

Respuesta.

```
200 OK
{
  "documento_id": "7f134334-8713-4f8a-84ff-cc45a55ed143",
  "total_paginas": 67,
  "secciones": [
    {
      "tipo": "referencias",
      "titulo_detectado": "7. Referencias bibliográficas",
      "pagina_inicio": 52,
      "pagina_fin": 67,
      "tiene_referencias": true
    }
  ],
  "tiene_seccion_referencias": true,
  "advertencia": null
}
```

Resultado. PASSED

test_el_markdown_extraido_queda_disponible

Descripción. El texto extraído del PDF queda disponible por la API como texto plano y conserva el contenido del documento.

Código.

```
def test_el_markdown_extraido_queda_disponible(self, api, documento):
    r = api.get(f"/api/v1/ingesta/{documento['documento_id']}/markdown")
```

```
assert r.status_code == 200
assert r.headers["content-type"].startswith("text/plain")
assert "GraphRAG" in r.text
```

Petición.

```
GET /api/v1/ingesta/7f134334-8713-4f8a-84ff-cc45a55ed143/markdown
```

Respuesta.

```
200 OK
<text/plain; charset=utf-8, 123133 bytes>
```

Resultado. PASSED

test_el_pdf_original_se_recupera

Descripción. El PDF original se recupera íntegro y con su tipo de contenido correcto: el sistema devuelve exactamente los bytes que se le entregaron.

Código.

```
def test_el_pdf_original_se_recupera(self, api, documento, pdf_tesis):
    r = api.get(f"/api/v1/ingesta/{documento['documento_id']}/pdf")
    assert r.status_code == 200
    assert r.headers["content-type"] == "application/pdf"
    assert len(r.content) == len(pdf_tesis)
```

Petición.

```
GET /api/v1/ingesta/7f134334-8713-4f8a-84ff-cc45a55ed143/pdf
```

Respuesta.

```
200 OK
<application/pdf, 730941 bytes>
```

Resultado. PASSED

test_la_extraccion_encontro_las_citas_del_documento

Descripción. Concluido el pipeline, el documento tiene citas extraídas y consultables por la API.

Código.

```
def test_la_extraccion_encontro_las_citas_del_documento(self, api, documento):
    r = api.get(f"/api/v1/grafos/{documento['documento_id']}/citas")
    assert r.status_code == 200
    assert r.json()["total_citas"] > 0
```

Petición.

```
GET /api/v1/grafos/7f134334-8713-4f8a-84ff-cc45a55ed143/citas
```

Respuesta.

```
200 OK
{
  "documento_id": "7f134334-8713-4f8a-84ff-cc45a55ed143",
  "total_citas": 44,
  "citas_parenteticas": 24,
  "citas_narrativas": 20,
  "citas": [
    {
```

```

"cita_id": "cfae37ca-aa83-43e6-ba2d-543eb74a48f2",
"texto_cita": "(Hernández et al., 2019; Perdomo et al., 2020)",
"tipo": "parentetica",
"pagina": 11,
"fragmento_oracion": "La integridad del discurso científico descansa sobre la validez
de sus
citas y referencias, las cuales funcionan como el cimiento probatorio que permite la
reproducibilidad y la inserción de nuevos hallazgos en el conocimiento existente. No
obstante,
investigaciones sobre la calidad de las tesis en el Perú revelan un escenario
preocupante: la
precisión bibliográfica suele alcanzar apenas un nivel \"suficiente\" o \"regular\",
con
deficiencias críticas en la exhaustividad y la exactitud de los datos (Hernández et
al., 2019;
Perdomo et al., 2020)",
"referen ... [32460 caracteres en total]

```

Resultado. PASSED

test_la_extraccion_encontro_las_referencias_del_documento

Descripción. Concluido el pipeline, el documento tiene referencias bibliográficas extraídas y consultables por la API.

Código.

```

def test_la_extraccion_encontro_las_referencias_del_documento(self, api, documento):
    r = api.get(f"/api/v1/grafos/{documento['documento_id']}/referencias")
    assert r.status_code == 200
    assert r.json()["total_referencias"] > 0

```

Petición.

```
GET /api/v1/grafos/7f134334-8713-4f8a-84ff-cc45a55ed143/referencias
```

Respuesta.

```

200 OK
{
  "documento_id": "7f134334-8713-4f8a-84ff-cc45a55ed143",
  "total_referencias": 40,
  "referencias": [
    {
      "referencia_id": "753b6614-3fd6-43cd-ac49-9909c83ce098",
      "autores": [
        "Ansari, S."
      ],
      "anio": 2026,
      "titulo": "Compound Deception in Elite Peer Review: A Failure Mode Taxonomy of 100
Fabricated
Citations at NeurIPS 2025 (arXiv:2602.05930)",
      "fuente": "arXiv",
      "doi": "10.48550/arXiv.2602.05930",
      "datos_incompletos": false,
      "campos_faltantes": [],
      "nivel_confianza": null
    },
    {
      "referencia_id": "e93ae24c-e43e-4812-94ac-0ae7109c6fc6",
      "autores": [
        "Antal, M.",
        "Buza, K."
      ],
    }
  ]
}

```

```

    "anio": 2025,
    "titulo": "Evaluating Open-Source LLMs in RAG Systems: A Benchmark on Diploma Theses
Abstracts
Using Ragas",
    "fuente": "Acta Universitatis Sapientiae, Informatica",
    "doi": "10.1007/s44427-025-00006-3",
    "datos_incompletos": false,
    "c ... [18700 caracteres en total]

```

Resultado. PASSED

test_el_resumen_del_grafo_concuerda_con_las_citas_y_referencias

Descripción. El resumen del grafo que alimenta la vista de conocimiento cuenta exactamente las mismas citas y referencias que devuelven sus respectivas listas: las dos vistas del mismo dato no se contradicen.

Código.

```

def test_el_resumen_del_grafo_concuerda_con_las_citas_y_referencias(self, api,
documento):
    """El grafo que se muestra al usuario y las listas que consulta cuentan lo
mismo."""
    doc = documento["documento_id"]
    citas = api.get(f"/api/v1/grafo/{doc}/citas").json()["total_citas"]
    refs =
api.get(f"/api/v1/grafo/{doc}/referencias").json()["total_referencias"]
    r = api.get(f"/api/v1/grafo/{doc}/resumen")
    assert r.status_code == 200
    assert r.json()["nodos_citas"] == citas
    assert r.json()["nodos_referencias"] == refs

```

Petición.

- 1) GET /api/v1/grafo/7f134334-8713-4f8a-84ff-cc45a55ed143/citas
- 2) GET /api/v1/grafo/7f134334-8713-4f8a-84ff-cc45a55ed143/referencias
- 3) GET /api/v1/grafo/7f134334-8713-4f8a-84ff-cc45a55ed143/resumen

Respuesta.

```

1) 200 OK
{
  "documento_id": "7f134334-8713-4f8a-84ff-cc45a55ed143",
  "total_citas": 44,
  "citas_parenteticas": 24,
  "citas_narrativas": 20,
  "citas": [
    {
      "cita_id": "cfae37ca-aa83-43e6-ba2d-543eb74a48f2",
      "texto_cita": "(Hernández et al., 2019; Perdomo et al., 2020)",
      "tipo": "parentetica",
      "pagina": 11,
      "fragmento_oracion": "La integridad del discurso científico descansa sobre la validez
de sus
citas y referencias, las cuales funcionan como el cimiento probatorio que permite la
reproducibilidad y la inserción de nuevos hallazgos en el conocimiento existente. No
obstante,
investigaciones sobre la calidad de las tesis en el Perú revelan un escenario
preocupante: la
precisión bibliográfica suele alcanzar apenas un nivel \"suficiente\" o \"regular\",
con
deficiencias críticas en la exhaustividad y la exactitud de los datos (Hernández et
al., 2019;

```

```

    Perdomo et al., 2020)",
    "referen ... [32460 caracteres en total]
2) 200 OK
{
  "documento_id": "7f134334-8713-4f8a-84ff-cc45a55ed143",
  "total_referencias": 40,
  "referencias": [
    {
      "referencia_id": "753b6614-3fd6-43cd-ac49-9909c83ce098",
      "autores": [
        "Ansari, S."
      ],
      "anio": 2026,
      "titulo": "Compound Deception in Elite Peer Review: A Failure Mode Taxonomy of 100
Fabricated
Citations at NeurIPS 2025 (arXiv:2602.05930)",
      "fuente": "arXiv",
      "doi": "10.48550/arXiv.2602.05930",
      "datos_incompletos": false,
      "campos_faltantes": [],
      "nivel_confianza": null
    },
    {
      "referencia_id": "e93ae24c-e43e-4812-94ac-0ae7109c6fc6",
      "autores": [
        "Antal, M.",
        "Buza, K."
      ],
      "anio": 2025,
      "titulo": "Evaluating Open-Source LLMs in RAG Systems: A Benchmark on Diploma Theses
Abstracts
Using Ragas",
      "fuente": "Acta Universitatis Sapientiae, Informatica",
      "doi": "10.1007/s44427-025-00006-3",
      "datos_incompletos": false,
      "c ... [18700 caracteres en total]
3) 200 OK
{
  "documento_id": "7f134334-8713-4f8a-84ff-cc45a55ed143",
  "total_nodos": 264,
  "nodos_autores": 180,
  "nodos_referencias": 40,
  "nodos_citas": 44,
  "total_relaciones": 306,
  "densidad_promedio": 1.16,
  "grafo_robusto": false,
  "citas_vinculadas": 42,
  "advertencia_densidad": "La densidad del grafo es 1.16 relaciones/nodo, por debajo del
mínimo
recomendado de 3. El documento puede tener pocas referencias o citas.",
  "error": null
}

```

Resultado. PASSED

7.2.2. TestTransicionesInvalidas

Lo que el sistema debe impedir. El orden de estas pruebas es el del ciclo de vida.

test_verificar_antes_de_confirmar_la_revision_es_rechazado

Descripción. Transición inválida: pedir la verificación externa antes de que el usuario confirme la revisión se rechaza con 400 y código ESTADO_INVALIDO.

Código.

```
def test_verificar_antes_de_confirmar_la_revision_es_rechazado(self, api,
documento):
    r = api.post(
        f"/api/v1/ingesta/{documento['documento_id']}/verificar",
        json={"referencia_ids": ["r1"]},
    )
    assert r.status_code == 400
    assert r.json()["detail"]["codigo"] == "ESTADO_INVALIDO"
```

Petición.

POST /api/v1/ingesta/7f134334-8713-4f8a-84ff-cc45a55ed143/verificar
cuerpo: {"referencia_ids": ["r1"]}

Respuesta.

```
400 Bad Request
{
  "detail": {
    "codigo": "ESTADO_INVALIDO",
    "mensaje": "El documento no está listo para verificación.",
    "accion_sugerida": "Espera a que la extracción termine antes de verificar."
  }
}
```

Resultado. PASSED

test_confirmar_la_revision_desbloquea_la_verificacion

Descripción. Transición válida: confirmar la revisión lleva el documento al estado listo_extraccion, que habilita la verificación externa.

Código.

```
def test_confirmar_la_revision_desbloquea_la_verificacion(self, api, documento):
    r = api.post(f"/api/v1/ingesta/{documento['documento_id']}/confirmar-
revision")
    assert r.status_code == 200
    assert r.json()["estado"] == "listo_extraccion"
```

Petición.

POST /api/v1/ingesta/7f134334-8713-4f8a-84ff-cc45a55ed143/confirmar-revision

Respuesta.

```
200 OK
{
  "documento_id": "7f134334-8713-4f8a-84ff-cc45a55ed143",
  "estado": "listo_extraccion"
}
```

Resultado. PASSED

test_confirmar_la_revision_dos_veces_es_rechazado

Descripción. Transición inválida: confirmar dos veces la revisión se rechaza con 400. La operación no es idempotente y el sistema lo declara explícitamente en lugar de repetir el efecto.

Código.

```
def test_confirmar_la_revision_dos_veces_es_rechazado(self, api, documento):
    r = api.post(f"/api/v1/ingesta/{documento['documento_id']}/confirmar-revision")
    assert r.status_code == 400
    assert r.json()["detail"]["codigo"] == "ESTADO_INVALIDO"
```

Petición.

POST /api/v1/ingesta/7f134334-8713-4f8a-84ff-cc45a55ed143/confirmar-revision

Respuesta.

```
400 Bad Request
{
  "detail": {
    "codigo": "ESTADO_INVALIDO",
    "mensaje": "El documento no está en fase de revisión.",
    "accion_sugerida": "Espera a que la extracción termine antes de confirmar."
  }
}
```

Resultado. PASSED

test_verificar_sin_seleccionar_referencias_es_rechazado

Descripción. Petición sin contenido útil: pedir la verificación con una lista vacía de referencias se rechaza con 400 y código SIN_REFERENCIAS.

Código.

```
def test_verificar_sin_seleccionar_referencias_es_rechazado(self, api, documento):
    r = api.post(
        f"/api/v1/ingesta/{documento['documento_id']}/verificar",
        json={"referencia_ids": []},
    )
    assert r.status_code == 400
    assert r.json()["detail"]["codigo"] == "SIN_REFERENCIAS"
```

Petición.

POST /api/v1/ingesta/7f134334-8713-4f8a-84ff-cc45a55ed143/verificar
cuerpo: {"referencia_ids": []}

Respuesta.

```
400 Bad Request
{
  "detail": {
    "codigo": "SIN_REFERENCIAS",
    "mensaje": "Debes seleccionar al menos una referencia para verificar.",
    "accion_sugerida": "Selecciona una o más referencias en la lista."
  }
}
```

Resultado. PASSED

test_confirmar_la_revision_de_un_documento_inexistente_es_rechazado

Descripción. Confirmar la revisión de un documento que no existe se rechaza con 400 y código ESTADO_INVALIDO, sin crear nada.

Código.

```
def test_confirmar_la_revision_de_un_documento_inexistente_es_rechazado(self, api,
documento_inexistente):
    r = api.post(f"/api/v1/ingesta/{documento_inexistente}/confirmar-revision")
    assert r.status_code == 400
    assert r.json()["detail"]["codigo"] == "ESTADO_INVALIDO"
```

Petición.

```
POST /api/v1/ingesta/850887b8-9e65-4a64-ad37-aef1a9895a91/confirmar-revision
```

Respuesta.

```
400 Bad Request
{
  "detail": {
    "codigo": "ESTADO_INVALIDO",
    "mensaje": "El documento no está en fase de revisión.",
    "accion_sugerida": "Espera a que la extracción termine antes de confirmar."
  }
}
```

Resultado. PASSED

test_los_veredictos_no_existen_antes_de_auditar

Descripción. Antes de ejecutar la auditoría no hay veredictos que consultar: la API responde 404 con código propio. Es el estado que la interfaz refleja como «Aún no hay veredictos» (figura 5).

Código.

```
def test_los_veredictos_no_existen_antes_de_auditar(self, api, documento):
    r = api.get(f"/api/v1/auditoria/{documento['documento_id']}/veredictos")
    assert r.status_code == 404
    assert r.json()["detail"]["codigo"] == "VEREDICTOS_NO_ENCONTRADOS"
```

Petición.

```
GET /api/v1/auditoria/7f134334-8713-4f8a-84ff-cc45a55ed143/veredictos
```

Respuesta.

```
404 Not Found
{
  "detail": {
    "codigo": "VEREDICTOS_NO_ENCONTRADOS",
    "mensaje": "No hay veredictos calculados para este documento.",
    "accion_sugerida": "Ejecuta primero POST /auditoria/{documento_id}/auditar."
  }
}
```

Resultado. PASSED

test_auditar_un_documento_sin_citas_no_produce_veredictos

Descripción. Auditar un documento sin citas no produce veredictos ni invoca al modelo de lenguaje: la API responde 404 con código SIN_CITAS.

Código.

```
def test_auditar_un_documento_sin_citas_no_produce_veredictos(self, api,
documento_inexistente):
    r = api.post(f"/api/v1/auditoria/{documento_inexistente}/auditar")
```

```
assert r.status_code == 404
assert r.json()["detail"]["codigo"] == "SIN_CITAS"
```

Petición.

```
POST /api/v1/auditoria/850887b8-9e65-4a64-ad37-aef1a9895a91/auditar
```

Respuesta.

```
404 Not Found
{
  "detail": {
    "codigo": "SIN_CITAS",
    "mensaje": "No se encontraron citas para auditar en este documento.",
    "accion_sugerida": "Verifica que la auditoría inicial se haya completado."
  }
}
```

Resultado. PASSED

8. Defecto detectado

La prueba `test_un_pdf_con_cabecera_valida_pero_cuerpo_danado_es_rechazado` falló, y su fallo corresponde a un defecto real del sistema, no a un error de la prueba.

Atributo	Descripción
Síntoma	Al cargar un PDF con cabecera %PDF válida pero cuerpo ilegible, la API responde 500 Internal Server Error.
Comportamiento esperado	422 Unprocessable Entity con código PDF_CORRUPTO, mensaje explicativo y acción sugerida, como con cualquier otro archivo no procesable.
Causa	validar_pdf solo comprueba los cuatro primeros bytes del archivo, de modo que un cuerpo dañado supera la validación. Después, extraer_texto abre el documento con PyMuPDF sin capturar su excepción: el error escapa como fallo no controlado y no llega a traducirse a PDFNoProcessableError.
Impacto	El usuario que sube un PDF truncado —una descarga interrumpida, una exportación a medias— recibe un error genérico sin indicación de qué hacer. Además, el PDF ya subido queda huérfano en el almacenamiento, porque su borrado solo ocurre en la rama del error controlado.
Detectabilidad	Las pruebas unitarias no pueden encontrarlo: la que cubre el rechazo por contenido no válido ejercita únicamente el camino de la cabecera, y pasa. El defecto solo se manifiesta al entrar por la API, con el archivo ya almacenado.
Corrección propuesta	Capturar la excepción de PyMuPDF en extraer_texto y traducirla a PDFNoProcessableError con código PDF_CORRUPTO, asegurando además el borrado del objeto subido. La prueba de esta suite queda como prueba de regresión.

La evidencia registrada en la ejecución es la siguiente:

```
Petición:  POST /api/v1/ingesta/cargar
           cuerpo: archivo=danado.pdf (<4105 bytes>, cabecera %PDF y cuerpo
ilegible)
Respuesta: 500 Internal Server Error
           <text/plain; charset=utf-8, 21 bytes>
Esperado:  422 Unprocessable Entity ·  codigo='PDF_CORRUPTO'
```

9. Capturas de la interfaz

Las capturas siguientes se tomaron con un navegador automatizado sobre la aplicación en funcionamiento, con el documento de prueba cargado durante esta misma ejecución (identificador 7f134334-8713-4f8a-84ff-cc45a55ed143). Muestran el estado real del sistema al terminar la suite: el documento procesado, sus citas y referencias extraídas, su grafo de conocimiento y la auditoría todavía sin ejecutar.

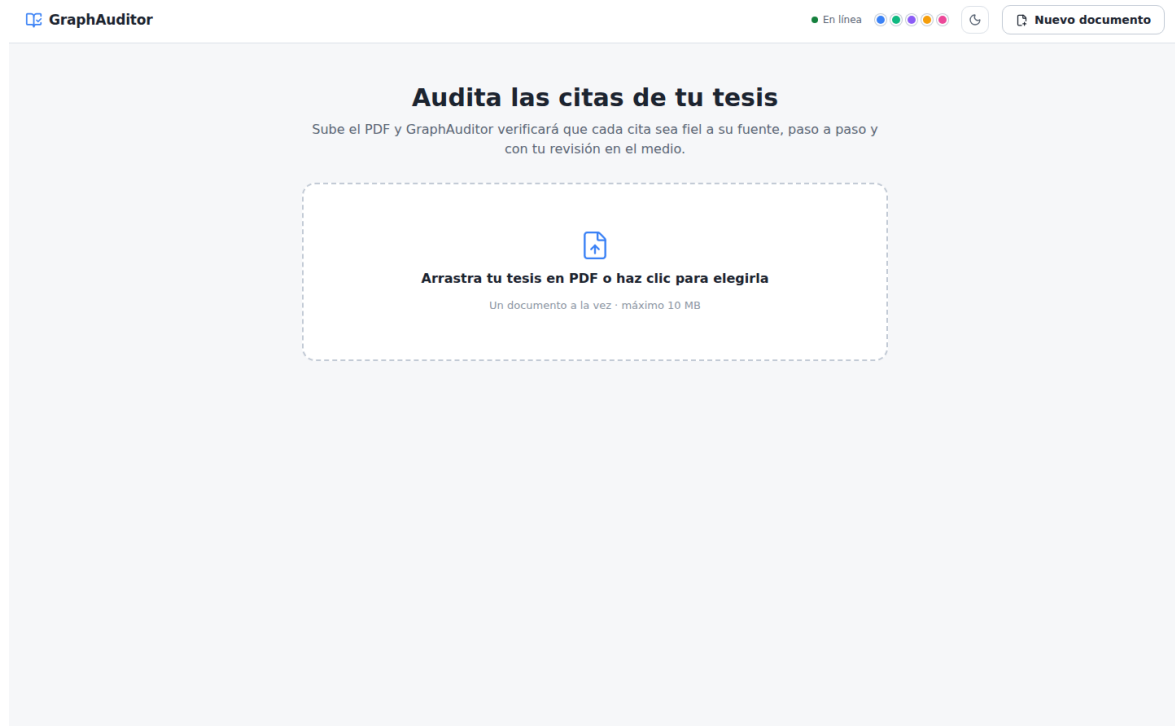


Figura 1. Pantalla de carga del documento.

GraphAuditor 7f134334-8713-4f8a-84ff-cc45a55ed143 En línea Nuevo documento

Revisa lo que detectamos
Corrige las citas y referencias antes de continuar: haz clic en una cartilla para ver el punto exacto del PDF donde aparece.

Revisión

Verificación

Grafo

Auditoría

Cierre

Revisa lo que detectamos

Re-vincular automáticamente

Revisión confirmada

Citas (44)

Referencias (40)

Ver referencias en el PDF

44 citas · 2 sin referencia

solo sin vincular

seleccionar visibles

+ Añadir cita

Parentética

pág. 11

Hernández et al. (2019)

TEXTO DE LA CITA

(Hernández et al., 2019; Perdomo et al., 2020)

ASEVERACIÓN DEL TESISTA

La integridad del discurso científico descansa sobre la validez de sus citas y referencias, las cuales funcionan como el cimiento probatorio que permite la

REFERENCIA VINCULADA

Hernández et al. (2019) — escribe para

PÁGINA

11

Eliminar

Guardar

Parentética

pág. 11

Wang et al. (2026)

TEXTO DE LA CITA

(Zhao et al., 2026)

ASEVERACIÓN DEL TESISTA

La irrupción de los Modelos de Lenguaje de Gran Tamaño (LLM) ha exacerbado esta problemática al introducir una escala sin precedentes de alucinaciones

UNIVERSIDAD PRIVADA ANTONOR ORREGO
FACULTAD DE INGENIERÍA
PROGRAMA DE ESTUDIO DE INGENIERÍA DE COMPUTACIÓN Y SISTEMAS

TESIS PARA OPTAR EL TÍTULO PROFESIONAL DE
INGENIERO DE COMPUTACIÓN Y SISTEMAS

Concordancia de un sistema GraphRAG con evaluación experta en detección de
inconsistencias en citas bibliográficas de tesis, UPAO 2024-2025

Línea de Investigación:
Robótica, automatización avanzada y sistemas inteligentes.
Sub Línea de Investigación: Sistemas Inteligentes
Autor: Alcalde Lavado Matias Felipe

Jurado evaluador:
Presidente: Apellidos, Nombres
Secretario: Apellidos, Nombres
Vocal: Asesor:
Cueva Chavez, Walter

Figura 2. Revisión humana: las 44 citas y 40 referencias extraídas del documento de prueba, en cartillas editables junto al PDF.

GraphAuditor 7f134334-8713-4f8a-84ff-cc45a55ed143 En línea Nuevo documento

Verificación

Carga

Revisión

Grafo

Auditoría

Cierre

Asocia las fuentes
La verificación automática usa solo el DOI de cada referencia (sin adivinar por título). Para las que no tienen DOI (3), importa tu colección de Zotero o sube el PDF.

Importar de Zotero

Verificar 37 por DOI

☒ **Ansari (2026)**
Compound Deception in Elite Peer Review: A Failure Mode Taxonomy of 100 Fabricated Citations at Ne... Subir PDF

☒ **Antal et al. (2025)**
Evaluating Open-Source LLMs in RAG Systems: A Benchmark on Diploma Theses Abstracts Using Ragas Subir PDF

☒ **Arévalo et al. (2021)**
Analysis of University STEM Students' Mathematical, Linguistic, Rhetorical-Organizational Assignment ... Subir PDF

☒ **Bakker et al. (2023)**
Evaluating the Accuracy of scite, a Smart Citation Index Subir PDF

☒ **Besrouer et al. (2025)**
SQuAI: Scientific Question-Answering with Multi-Agent Retrieval-Augmented Generation (Versión 1) Subir PDF

☒ **Cabezas-Clavijo et al. (2026)**
Assessing the Performance of 8 AI Chatbots in Bibliographic Reference Retrieval: Grok and DeepSeek ... Subir PDF

☐ **Teplitskiy et al. (2025)**
The Noisy Path from Source to Citation: Measuring How Scholars Engage with Past Research sin DOI Subir PDF

☒ **Cheng et al. (2025)**
Artificial intelligence-assisted academic writing: Recommendations for ethical use Subir PDF

☒ **Dunford et al. (2024)**
Using automated analysis of the bibliography to detect potential research integrity issues Subir PDF

Figura 3. Selección de las referencias que se verificarán contra CrossRef y Unpaywall.

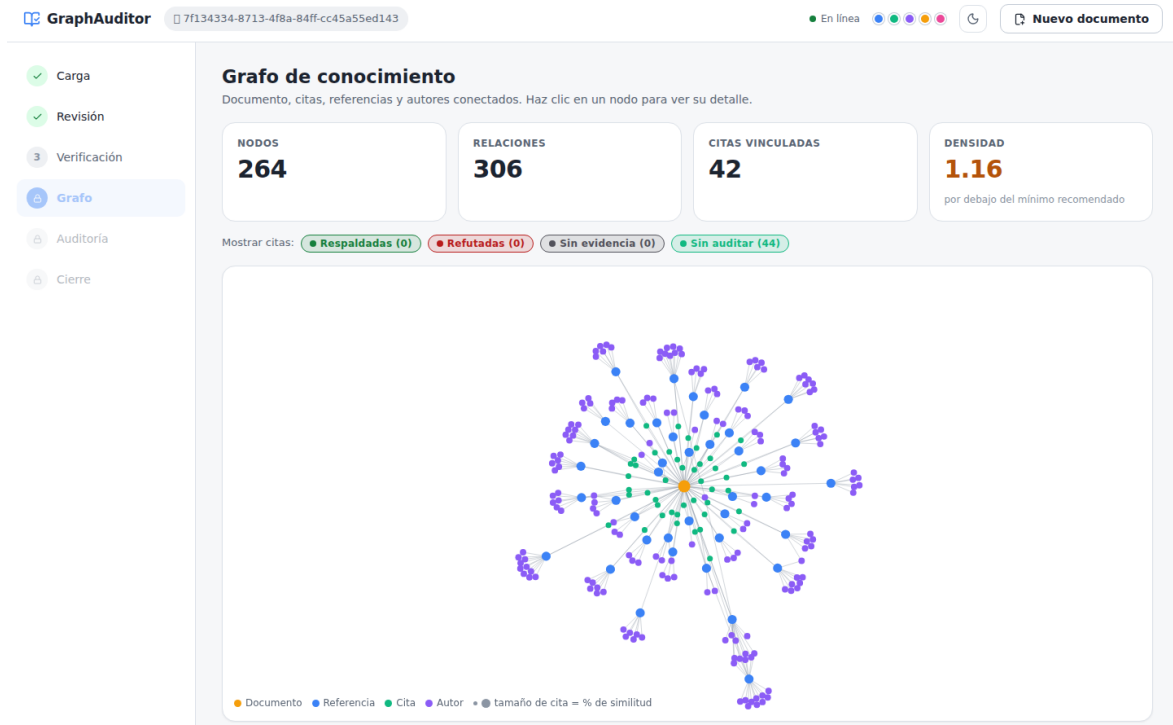


Figura 4. Grafo de conocimiento del documento: 264 nodos y 306 relaciones, con 42 citas vinculadas a su referencia.

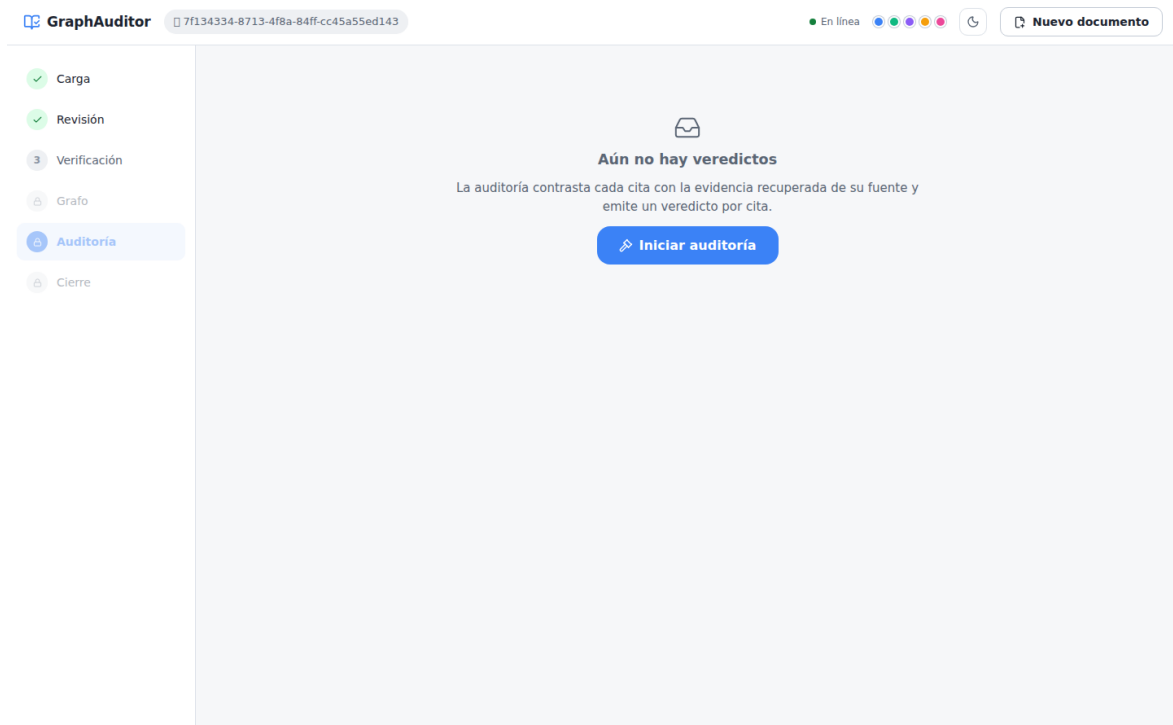


Figura 5. Auditoría antes de ejecutarse: la interfaz declara que aún no hay veredictos, en coherencia con el 404 que devuelve la API.